Secure System Development Project Report

Team:

Sergey Aitov (CBS-02) - s.aitov@innopolis.university
Dmitriy Prokopyev (CBS-02) - d.prokopyev@innopolis.university
Alexander Rozanov (CBS-02) - al.rozanov@innopolis.university
Egor Dushin (CBS-02)- e.dushin@innopolis.university

Introduction

Modern web applications often keep credentials in memory throughout the process lifecycle. This is convenient for development, but it poses a security risk: passwords, tokens, and encryption keys could be leaked if memory read access is compromised. An attacker can accomplish this with sufficient precision by abusing a buffer over-read or a similar vulnerability.

One solution is to **isolate** the component responsible for business logic from credential handling. This would require a proxy service that securely obtains and uses the credentials. The main application would **delegate** requests that require credentials to the proxy. The proxy, in turn, would retrieve the secrets from a specialized storage and use them only when performing data operations.

> **Secretless Architecture**
>
> Similarities to secretless architecture often implemented with **Conjur** sidecars is intentional. The chosen architecture has similar tradeoffs, but can offer extra beneficial properties and simplicity.

Thus, the goal of our work was to implement a proof-of-concept architecture in which database secrets are isolated from the main application, and data access is performed through a separate internal component with a smaller attack surface. This reduces the risk of key compromise during an attack on the web service, limits potential damage, and demonstrates the practical application of the principles of secure design, least privilege, and secret isolation.

---

2. Methods

2.1. Original app Movie Night

The project was based on the lightweight web app **Movie Night**, developed by the team for a project on **Software Quality, Reliability, and Security**. The app is a service for dorm residents that allows them to organize movie nights in common rooms.

2.1.1. Key features:

- User registration and login;
- JWT authentication and logout via token revocation;
- Creating movie screening suggestions;
- Checking schedules and conflicts by room and time;
- Voting within a conflicting group;
- Deterministic winner selection;
- Snack reactions for the selected screening;
- Displaying the homepage, offer catalog, and weekly schedule;
- Streamlit interface for manual use and exploratory testing.

2.1.2. Application Tech Stack

Backend:

- **Python 3.11+** - primary development language;
- **FastAPI** - REST API and OpenAPI/Swagger documentation implementation;
- **SQLAlchemy 2.x** - ORM and database management;
- **SQLite** - local database for persistent storage;
- **Pydantic / pydantic-settings** - request-response schemas and application configuration;
- **PyJWT** - JWT token issuance and verification;
- **pwdlib / Argon2id** - secure hashing of user passwords.

Frontend:

- **Streamlit** - a simple user interface for demonstration and manual testing;
- **requests** - interaction between the frontend and the backend API.

Quality and verification tools:

- **Poetry** - dependency management and command execution;
- **pytest / pytest-cov** - testing and coverage;
- **flake8** - static style checking and basic errors;
- **bandit** - static security analysis of Python code;
- **radon** - code complexity assessment;
- **locust** - load testing;

2.1.3. API endpoints Applications

The main application provides a **REST API** under the `/api/v1` prefix. The most important endpoints are:

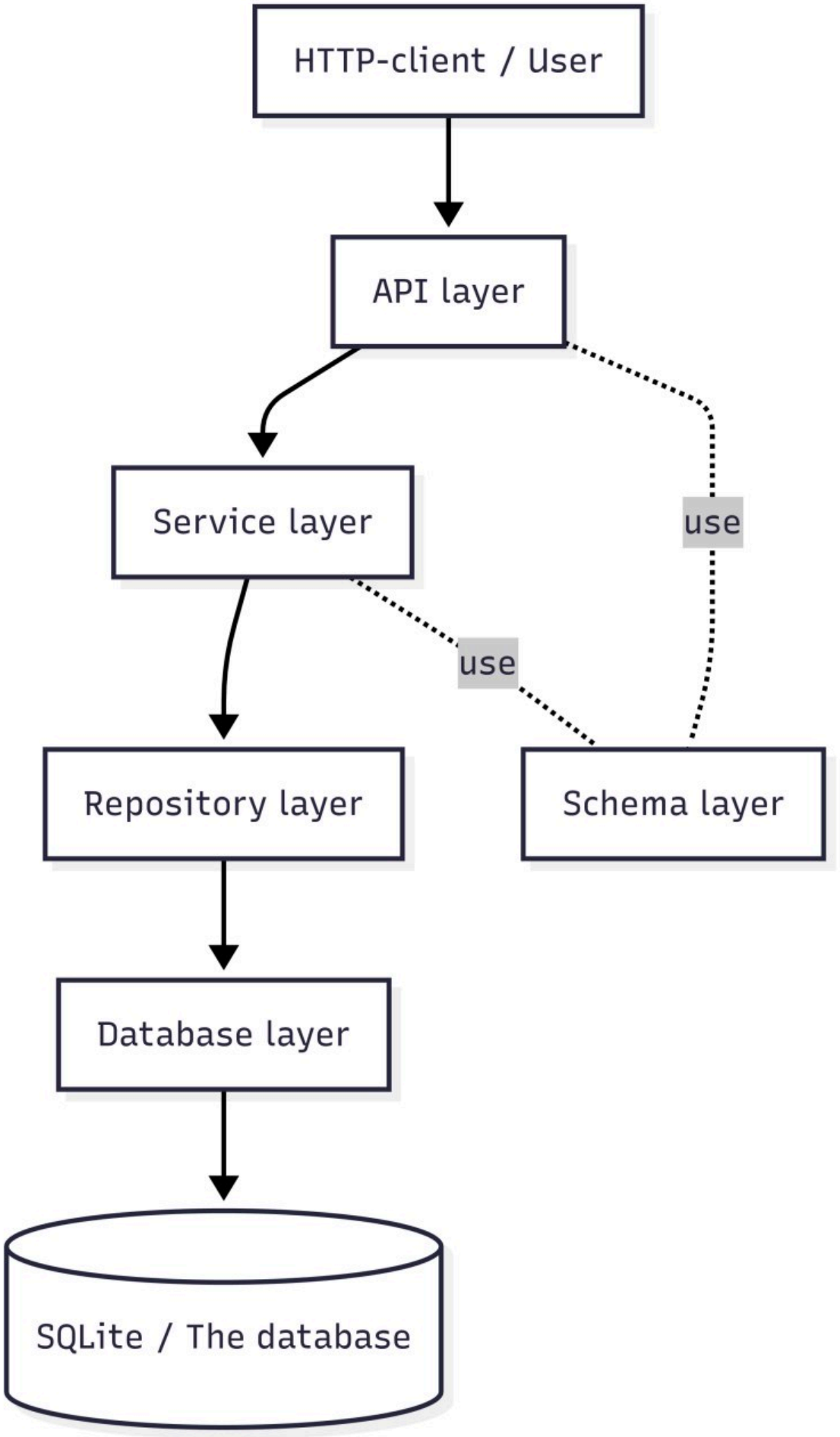
- **System endpoints:**
  - `GET /` - basic application status;
  - `GET /health` - backend service health check;
  - `GET /docs` - Swagger UI;
  - `GET /redoc` - ReDoc documentation.
- **Authentication:**
  - `POST /api/v1/auth/login` - user login;
  - `POST /api/v1/auth/register` - user registration;
  - `POST /api/v1/auth/logout` - revoke the current JWT;
  - `GET /api/v1/auth/me` - get the current user.
- **Basic application data:**
  - `GET /api/v1/home` - an aggregated home page with offers, conflicts, votes, and available actions;
  - `GET /api/v1/catalog` - a catalog of all offers;
  - `GET /api/v1/schedule` - a weekly schedule of rooms in fixed two-hour slots.
- **Proposal Lifecycle:**
  - `POST /api/v1/proposals` - create a review proposal;
  - `DELETE /api/v1/proposals/{proposal_id}` - delete a proposal by its owner.
- **Voting:**
  - `POST /api/v1/proposals/{proposal_id}/votes` - add a vote;
  - `DELETE /api/v1/proposals/{proposal_id}/votes` - delete a vote.
- **Reactions:**
  - `POST /api/v1/proposals/{proposal_id}/reactions` - add a snack reaction;
  - `DELETE /api/v1/proposals/{proposal_id}/reactions/{category}` - delete a reaction.

2.1.4. Initial Database Model

In the original ***Movie Night*** architecture, the backend was organized according to the classic layered model:

- The API layer received HTTP requests and called services;
- The service layer implemented business rules;
- The repository layer encapsulated SQLAlchemy queries;
- The database layer was responsible for ORM models, sessions, and database initialization;

- The schema layer described `pydantic` requests and responses.



The database contained several main entities:

- `User` - users;
- `Proposal` - movie suggestions;
- `Vote` - user votes;
- `FoodReaction` - reactions to the selected movie;
- `RevokedToken` - revoked JWT tokens for logout.

2.1.5. Risk of Secret Leaks from the Application Runtime Context

In the original architecture, the public backend simultaneously processed HTTP requests and directly accessed SQLite via `SQLAlchemy`. With the addition of SQLCipher, this process would be responsible for obtaining the database key, so the secret could leak into memory, environment variables, or container configuration. In the event of an RCE, LFI, debug information leak, or process dump, an attacker could extract the key and access the data.

2.2. Implementation of database secret isolation

As part of the work, a PoC architecture was implemented aimed at isolating the database secret from the main web application. The main idea was to keep the same user logic of the application, but to bring direct work with the encrypted SQLite database and its key into a **separate internal component**.

2.2.1. Operation of the main application

The main application retained its role as a public entry point and continued to process user HTTP requests. From the point of view of the external API, the set of main user sections has not changed: the application still has routes for authentication, the catalog, the main page, suggestions, reactions, schedules and voting.

The key change occurred at the level of data access. In the original version, the backend directly created an `SQLAlchemy` session and worked with SQLite via `SessionLocal`. In the new architecture, this dependency has been **replaced** by `SQLiteProxyClient`, which accesses the internal proxy component via HTTP.

2.2.2. Adding a proxy component for accessing the database

To separate the main application from direct access to the database, a separate `sqlite-proxy` component was added. This is an internal FastAPI service that is responsible for operations with an encrypted SQLite database: obtaining a key, opening the database, initializing the schema, and performing operations required by the application.

Instead of a universal SQL interface, proxy provides a limited set of **internal endpoints**, grouped by functional areas:

- `/internal/auth/*` - operations with users and revoked tokens;
- `/internal/proposals/*` - create and delete proposals;
- `/internal/votes/*` - adding and deleting votes;
- `/internal/reactions/*` - adding and removing reactions;
- `/internal/reads/*` - getting read models for home, catalog, and schedule.

Advantages of the approach:

- `sqlite-proxy` has a lower attack surface both at the interface level and at the component level itself: it is not a public entry point, does not serve the user web interface, accepts requests only from the closed Docker network where the main application is located, contains less application logic and provides a limited set of business-operations instead of random SQL access;
- A limited set of acceptable operations reduces the risk of proxy API abuse: it is more difficult for an attacker to exploit predefined business operations than a universal database gateway;
- `sqlite-proxy` is easier to review, test, and strengthen, since its area of responsibility is limited to direct database operations;
- All database accesses are concentrated in one small service, which simplifies the subsequent addition of monitoring, auditing and alerting for sensitive operations;
- The proxy component can additionally be protected independently of the main application: restrict network access, add service-to-service authentication, logging, rate limiting, and separate control rules.

2.2.3. Adding Vault

HashiCorp Vault has been added as a component for generating and storing the encryption key. In the current PoC, it is used in the development/bootstrap scenario: a random 32-byte key is generated through Vault Transit, which is then passed to Conjur for further use by the proxy component.

2.2.4. Adding Conjur

Conjur Open Source has been added to control access to the database key. In this architecture, Conjur is used not just as a secret storage location, but as a component that provides machine identity and policy-based access control for services.

A separate policy for SQLite proxy has been created in Conjur. It describes the namespace `sqlite-policy/`, the host identity `sqlite-policy/database-proxy`, the variable `sqlite-policy/sqlite/key` and the permissions to access it. According to the Conjur model, access is prohibited by default, and key reading is allowed only by the explicitly specified machine identity proxy component.

Due to this, the main application does not get access to the variable with the database key. The right to read the secret is not tied to a public backend service, but to a separate internal component that directly works with an encrypted SQLite database.

Synchronization of the SQLite encryption key between Vault and Conjur was also prepared. In the implemented PoC, the proxy can initiate the receipt of random key through Vault Transit and write the received SQLCipher key to the Conjur variable `sqlite-policy/sqlite/key`. After that, when working with the database, the proxy receives the key through Conjur using its machine identity.

2.2.5. Preparation of the verification methodology

To confirm the operability of the proposed PoC, a verification methodology was developed to demonstrate the key property of the architecture - isolation of the database secret from the main application.

1. Ensure the presence of the encryption key in **HashiCorp Vault**:

```
docker compose -f docker-compose.yaml --env-file .env.example exec vault vault kv get -
field=encryption_key secret/movienight/sqlite/key
```

2. Ensure valid extraction of the encryption key from **Conjur Open Source**:

```
docker compose -f docker-compose.conjur.yaml exec -T client conjur variable get -i sqlite-
policy/sqlite/key
```

3. Ensure the key **cannot be obtained** from the dump of the main app's memory.

```
APP_PID=$(docker inspect -f '{{.State.Pid}}' sqrs-movienight-app-1)
PROXY_PID=$(docker inspect -f '{{.State.Pid}}' sqrs-movienight-sqlite-proxy-1)
echo "app=$APP_PID  proxy=$PROXY_PID"

mkdir -p /tmp/dumps
sudo gcore -o /tmp/dumps/app "$APP_PID"
sudo gcore -o /tmp/dumps/proxy "$PROXY_PID"

sudo strings -a /tmp/dumps/app.$APP_PID | grep -c "$KEY"
sudo strings -a /tmp/dumps/proxy.$PROXY_PID | grep -c "$KEY"
sudo rm /tmp/dumps/app.$APP_PID /tmp/dumps/proxy.$PROXY_PID
```

> **Note**
>
> Verifying the presence of the key in the proxy's memory dump is necessary because it guarantees that the procedure **can** extract the key if it was present in the memory dump.

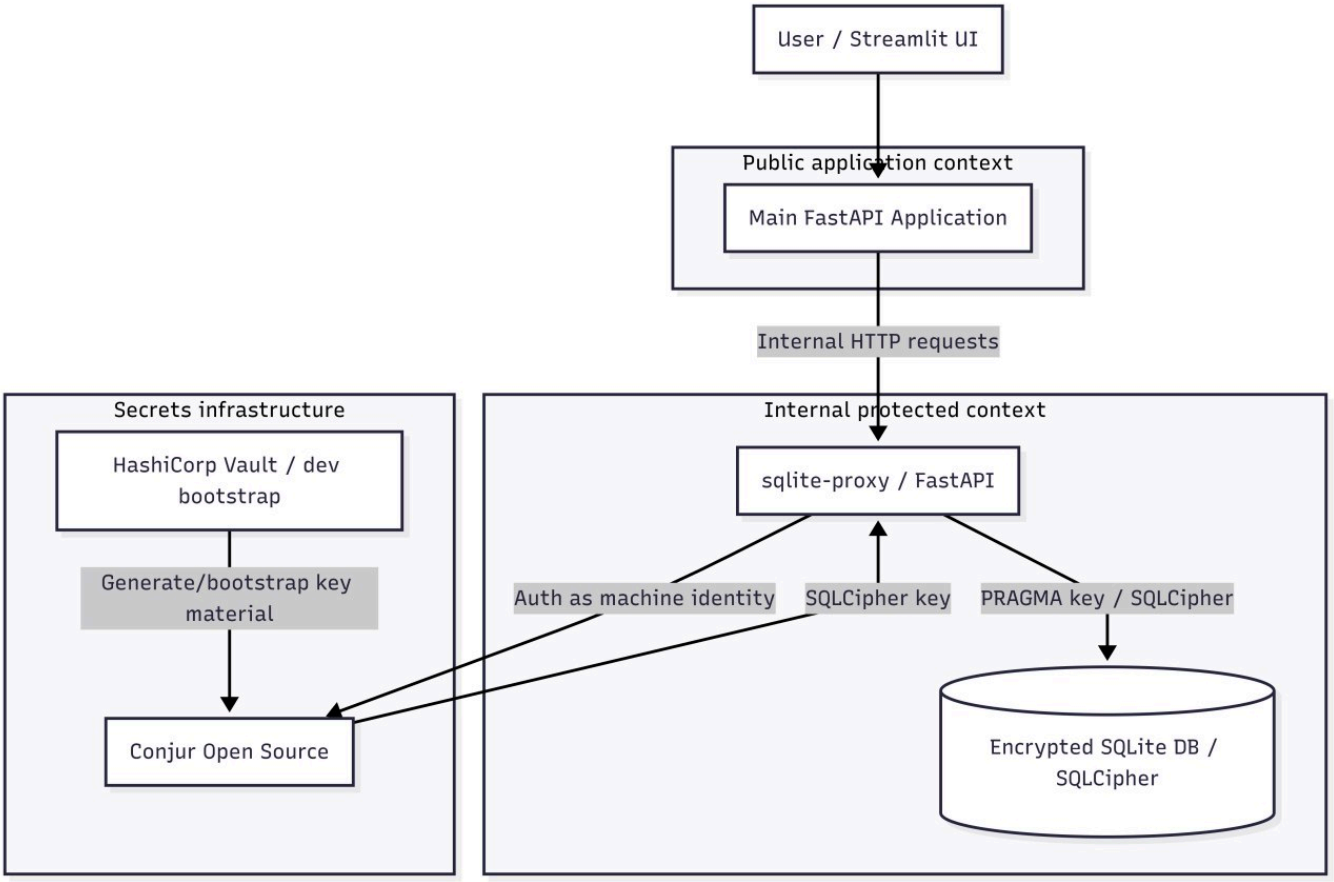
3. Results

3.1. The current operating principle of the application

After implementing the improvements, the application operates according to the **following logic**:

1. The user interacts with the public FastAPI backend via the REST API or Streamlit frontend.
2. The main application performs the business logic.
3. When a data operation is required, the main backend does not open SQLite directly, but calls `sqlite-proxy` via the internal HTTP API, which recognizes the backend **because of the private network**.
4. `sqlite-proxy` authenticates with Conjur using its machine identity.
5. Conjur issues the `SQLCipher` key only to the authorized proxy component.
6. `sqlite-proxy` opens the encrypted SQLite database, performs the required operation, and returns the result to the main application.
7. The main application generates a response to the user without accessing the database key.

The final architecture can be represented as follows:



3.2. Results of the PoC health check

As part of the verification, the operability of the main components of the PoC architecture and the correctness of the scenario for isolating the database secret from the main application were **confirmed**.

The SQLite encryption key is present in the Vault

Extracting the encryption key from Vault confirmed the presence of key used to protect the SQLite database. This shows that the secret generation and management component is working correctly.

The encryption key is extracted through Conjur Open Source

Extraction of the encryption key from Conjur confirmed that the `SQLCipher` key is attained through the machine identity mechanism and policy-based access control. This means that the secret is accessed through a dedicated access control loop, rather than directly from the main application.

The key is missing from the main application's memory

According to the results of analyzing the snapshot of the RAM of the backend process using `strings`, the value of the `SQLCipher` key was not detected. More importantly, the same procedure successfully extracted the key from the proxy. This confirms that the main application **does not contain the encryption key**, but the protected proxy does.

Thus, the verification showed that the implemented PoC works correctly: the key is created and accessible through the secret management loop, the `sqlite-proxy` can use it to work with an encrypted database, and the main backend process remains isolated from the secret.

---

4. Discussion

4.1. Advantages of the implemented approach

1. **Isolation of the database secret from the main application:**

The main advantage of the implemented PoC is that the main backend no longer requests the SQLCipher key and does not use it to open the SQLite database. This reduces the risk of compromising the secret during attacks on the main web service, including reading the process memory, leaking environment variables, debug information, or container configuration.

2. **Reducing the attack surface of the component handling the key:**

The database key is not used by the public backend application, but by a separate `sqlite-proxy`. This component is not an external entry point, does not serve the user interface, and only accepts requests from the private Docker network where the main application resides. This reduces the attack surface of the component handling the secret.

3. **Limited internal API instead of arbitrary database access:**

`sqlite-proxy` does not provide a universal SQL interface and does not allow arbitrary database queries. Instead, it implements a limited set of internal endpoints for specific application operations: users, suggestions, votes, reactions, and read models. This reduces the risk of abuse of the proxy component, as an attacker has fewer permitted actions.

4. **Preserving the external API and user logic of the application:**

The external endpoints of the main application remained the same, so the user experience remained unchanged. The change was made internally: instead of directly accessing SQLite, the main backend began using the `SQLiteProxyClient` and the proxy component's internal HTTP API. This demonstrates that architectural enhancements can be implemented without completely redesigning the user API.

5. **Division of responsibilities between components:**

In the new design, the main backend is responsible for the public API and application logic, `sqlite-proxy` is responsible for direct operations with the encrypted database, Vault is responsible for key management, and Conjur is responsible for machine identity and policy-based access control. This separation makes the architecture more understandable and manageable.

6. **A more convenient point for monitoring and auditing:**

All direct database operations are concentrated in a single, small service. This simplifies future additions of monitoring, auditing, and alerting: instead of monitoring multiple areas of the main application, you can focus on the `sqlite-proxy`, which handles sensitive data operations.

7. **Simplifying the review of the component that handles secrets:**

Since `sqlite-proxy` has a narrower scope of responsibility and less application logic, it is easier to thoroughly analyze, test, and strengthen. This is especially important for the component that handles the database key and performs operations on encrypted storage.

4.2. Limitations of the Implemented Approach

1. **PoC is not a fully production-ready implementation:**

The implemented scheme demonstrates the architectural idea of database secret isolation, but does not cover all production requirements. A production approach would require additional measures: full Vault configuration, secure token management, auditing, redundancy, centralized monitoring, and stronger authentication between services.

2. **Docker host compromise remains a critical scenario:**

If an attacker gains full control of the Docker host, they can potentially access containers, the file system, network communications, or process memory. In this case, isolation between the main application and `sqlite-proxy` can be bypassed.

3. **Compromising `sqlite-proxy` leads to the risk of key compromise:**

`sqlite-proxy` is the component that legitimately receives the `SQLCipher` key and opens the encrypted database. Therefore, if an attacker can execute code within the proxy component or read its memory, they can access the key. This architecture reduces the attack surface drastically, but does not eliminate the risk of an attack on the proxy itself.

4. **A compromise of Vault or Conjur violates the secret access model:**

Vault and Conjur are critical components of the secret management chain. If an attacker gains access to them, they can impact key generation, storage, issuance, or access policy. Therefore, these components require separate hardening, monitoring, access control, and secure operation.

5. **In the current PoC, Vault is used in a dev/bootstrap scenario:**

In the demo implementation, Vault is used to generate key material via Transit, but is run in dev mode. This mode is convenient for a training setup, but is not suitable for a production environment, as it does not provide full resiliency, secure storage, access management, or industrial-grade key lifecycle management.

6. **Main application authentication is based primarily on network isolation:**

In the current implementation, `sqlite-proxy` accepts requests from the private Docker network where the main application resides. This reduces the attack surface, but does not provide strong cryptographic authentication of the service. In a more strict environment, mTLS, signed requests, a service token, or another explicit service-to-service authentication mechanism should be added.

7. **The presence of an internal proxy API creates a new component that requires maintenance:**

Adding `sqlite-proxy` complicates the architecture: it introduces an additional service that must be launched, configured, updated, tested, and monitored. This increases operational complexity compared to direct backend application access to SQLite.

8. **The proxy component does not eliminate business logic errors:**

A limited set of endpoints reduces the risk of arbitrary database access, but does not protect against errors in the business operations themselves. If an endpoint is implemented incorrectly, for example, improperly validates user rights or allows restrictions to be bypassed, this error can still be exploited by an attacker.

9. **Secret isolation does not equal complete data protection:**

Even if the key is not in the main application's memory, an attacker who gains control of the public backend can attempt to exploit legitimate application functions or internal proxy calls within the available business logic. Therefore, key isolation mitigates one specific class of risks, but does not replace comprehensive protection of APIs, authorization, and data validation.

10. **Auditing and observability require further development:**

The architecture creates a convenient monitoring point `sqlite-proxy`, but the mere fact of centralizing operations doesn't necessarily mean full auditing. For practical implementation, detailed logging of sensitive operations, event correlation, alerts for abnormal behavior, and monitoring of Conjur/Vault calls need to be added.

11. **The verification methodology demonstrates a key scenario, but does not cover all possible attacks:**

Checking for the absence of a key in the main application's memory confirms the basic idea of the PoC, but it does not prove the security of the entire system. Additional tests would be required for network isolation bypass, proxy API abuse, token compromise, authorization errors, and the resilience of secret-management components.

12. **The solution is not feasible for all types of applications:**

For small or low-risk applications, such separation may be excessive due to the additional complexity. This approach is most useful in scenarios where it is especially important to reduce the risk of key leakage from a public backend process and centralize control over direct access to data.

4.3. Difficulties encountered

- Brittle naming and versioning differences in **HashiCorp Vault** and **Conjur Open Source** required a significant debugging effort to connect reliably.
- Standard `docker-compose` dependency-based ordering did not suffice for subtle initialization procedures carried out by separate processes.
- Inclusion of `cyberark/conjur` in a separate docker compose file lead to networking difficulties and extra debugging effort.

---

5. Conclusion

As a result, a **PoC** architecture was implemented that isolates the `SQLCipher` key from the main web application: the public backend retained the external API and business logic, but no longer manages SQLite encryption key. Vault was used to generate and store the encryption key, and Conjur was used for machine identity and policy-based access control.

Commands used for validation **confirmed** that the PoC is working correctly and the key is **not found** in the memory of the main backend process. At the same time, the solution remains a demo and requires improvements for the production environment, including enhanced service-to-service authentication, auditing, and protection of secret-management components.